1. Introduction

Modern semiconductor design includes stages such as RTL design, synthesis, verification, and timing analysis. RTL design describes digital circuits using Hardware Description Languages like Verilog or SystemVerilog and serves as the starting point of the ASIC design flow. Errors at this stage can affect later stages of the design process.

Designers often face issues such as incorrect assignments, missing reset logic, and incomplete conditions, which may lead to synthesis errors or functional failures. Static Timing Analysis (STA) is used to verify that circuits meet timing requirements, while verification ensures that the design functions correctly before fabrication.

Since these processes involve large reports, simulations, and repetitive tasks, Artificial Intelligence (AI) and Large Language Models (LLMs) are being explored to automate RTL generation, verification, and timing analysis, improving efficiency in the design flow.

2. Common problems

2.1 Common RTL Design Issues

1 Latch Inference

Latch inference occurs when signals are not assigned values under all possible conditions in a combinational logic block. When this happens, synthesis tools infer a latch to hold the previous value of the signal.

This usually occurs when conditional statements lack an else clause or when case statements do not cover all input possibilities. Unintentional latch creation can lead to unpredictable timing behavior and increased design complexity.

Example:

```
always @(*) begin
   if(a)
      y = b;
end
```

This code does not assign a value to y when a is false, resulting in latch inference.

---

2 Blocking vs Non-Blocking Assignment Misuse

Verilog provides two types of assignments: blocking (=) and non-blocking (<=). Blocking assignments execute sequentially, while non-blocking assignments execute concurrently.

Using blocking assignments in sequential logic may cause simulation behavior that differs from actual synthesized hardware. Therefore, non-blocking assignments should be used inside clocked always blocks.

---

3 Multiple Drivers on a Signal

Multiple drivers occur when more than one procedural block attempts to assign values to the same signal. This results in conflicting drivers and may lead to synthesis errors or undefined hardware behavior.

To prevent this issue, signals should typically be driven by a single always block.

---

4 Incomplete Case Statements

Case statements that do not include all possible input combinations may cause unintended latch inference. Including a default case ensures that outputs are assigned under all conditions.

---

5 Width Mismatch

Width mismatch occurs when signals with different bit widths are assigned to each other. This may lead to unintended truncation or sign extension, affecting the correctness of the design.

Proper signal declarations and explicit bit extensions should be used to prevent such problems.

---

6 Missing Reset Logic

Registers without reset conditions may power up in undefined states. Including synchronous or asynchronous reset logic ensures predictable system startup behavior.

---

7 Combinational Loops

Combinational loops occur when signals feed back into themselves without passing through a clocked element. These loops cause synthesis tools to fail or produce unstable circuits.

---

8 High Fanout Signals

Signals that drive a large number of loads are referred to as high fanout signals. These signals can create timing delays and routing congestion during physical design.

---

9 Uninitialized Registers

Registers that are not properly initialized may hold unknown values at startup, which may propagate through the design and cause functional errors.

---

10 Clock Domain Crossing (CDC) Issues

Clock domain crossing problems occur when signals move between modules operating on different clock signals. Without proper synchronization techniques, this may cause metastability and data corruption.

2.2Common Static Timing Analysis Problems

1. Setup Timing Violation
Occurs when data arrives too late before the clock edge.

Example:
Slack: -0.20 ns
Startpoint: reg_A
Endpoint: reg_B

Challenge:
Engineers must manually trace long combinational paths to identify the source of delay.

---

2. Hold Timing Violation
Occurs when data arrives too early after the clock edge.

Example:
Hold Slack: -0.05 ns

Challenge:
Finding the shortest path causing the violation can be difficult.

---

3. Clock Skew Issues
Clock skew refers to the difference in arrival time of the clock signal at different registers.

Challenge:
Large clock trees make debugging difficult.

---

4. Large Timing Reports
Static Timing Analysis (STA) tools generate reports containing thousands of timing paths.

Challenge:
Engineers must manually locate the worst negative slack paths.

---

5. Incorrect Timing Constraints
Timing analysis relies on constraints defined in SDC files.

Example:

create_clock -period 10 [get_ports clk]

Challenge:
Incorrect constraints can produce misleading timing results.

---

6. Missing False Path Constraints
Some paths should not be analyzed for timing.

Example:

set_false_path

Challenge:
Engineers must manually identify these paths.

---

7. Multicycle Path Errors
Some operations require multiple clock cycles to complete.

Example:

set_multicycle_path 2

Challenge:
Missing multicycle constraints can lead to unnecessary timing violations.

---

8. Long Critical Paths
Large combinational blocks may create long propagation delays.

Challenge:
Designers must manually insert pipeline registers to reduce delay.

---

9. Clock Domain Crossing Issues
Signals may transfer between different clock domains.

Challenge:
Detecting unsafe crossings can be difficult.

---

10. Timing Closure Iterations
Achieving timing closure often requires multiple design iterations.

Challenge:
Each iteration requires re-running synthesis and Static Timing Analysis.

2.3 Common Verification Problems

1. Missing or Incomplete Testbenches

Many RTL modules are created without proper testbenches.

Example:

```
module adder(
 input [3:0] a,
 input [3:0] b,
 output [4:0] sum
);
assign sum = a + b;
endmodule
```

Challenge: Engineers must manually write testbenches for each module

2. Poor Functional Coverage

Simulation tests often do not cover all possible input scenarios.

Example problem:

Corner cases like overflow or boundary conditions may not be tested.

Challenge: Missing coverage can lead to hidden functional bugs.

3. Difficult Simulation Debugging

Simulation logs can contain thousands of lines of output.

Example log:

Error: Expected sum=8 but received sum=7

Challenge: Engineers must manually trace signals to find the root cause.

4. Waveform Analysis is Manual

Debugging requires opening waveform viewers like GTKWave.

Engineers must manually inspect signals such as:

- Clock
- reset
- input values
- output signals

Challenge: Large waveforms are difficult to analyze.

5. Random Test Failures

Randomized testbenches may produce intermittent failures.

Example:

Simulation fails once but passes when rerun.

Challenge: Hard to reproduce the same bug again.

6. **Assertion Writing Complexity**

SystemVerilog assertions help detect errors during simulation.

Example:

assert property (a + b == sum);

Challenge: Writing assertions for every module is time-consuming.

7. **Poor RTL Documentation**

Many RTL modules lack proper documentation explaining their functionality.

Challenge: Verification engineers must read and understand the RTL before writing tests.

8. **Regression Testing Overhead**

Large designs require running thousands of test cases.

Challenge: Regression testing can take several hours or days.

9. **Testbench Maintenance**

Whenever the RTL design changes, the testbench may break.

Example:

Signal width changed from [3:0] to [7:0]

Challenge: Maintaining compatibility between RTL and testbench requires additional effort.

10. **limited Automation in Debugging**

Most debugging tasks are manual.

Engineers must:

- read simulation logs
- analyze waveform signals
- identify failing modules

Challenge: Low productivity due to repetitive tasks.

3. AI Training using LLM

3.1 RTL Generation Using Large Language Models

Recent advancements in artificial intelligence have enabled the use of Large Language Models (LLMs) to assist in hardware design tasks. These models can generate RTL code based on textual prompts provided by the designer.

In this study, we evaluated the capability of LLM-based systems to generate synthesizable RTL modules. Several prompts were provided to the model to generate common digital components such as counters, arithmetic units, and finite state machines.

Examples of generated modules include:

- 4-bit Arithmetic Logic Unit (ALU)
- Parameterized FIFO
- Traffic Light Controller FSM
- 8-bit Counter

The generated RTL code was analyzed to determine whether it followed synthesizable coding practices and whether it introduced common RTL design issues.

Observed Limitations in LLM-Generated RTL

Although LLMs are capable of generating syntactically correct Verilog code, several limitations were observed during evaluation.

Common issues include:

- Missing default conditions in case statements
- Improper use of blocking assignments
- Lack of reset logic
- Bit-width mismatches

These issues indicate that while LLMs can assist in rapid RTL generation, human verification remains essential to ensure correct hardware implementation.

3.2 AI-Based Timing Report Parsing

Timing reports generated by STA tools contain large amounts of textual data. Engineers often spend significant time manually inspecting these reports. AI and large language models can automate the analysis and highlight the most critical issues.

Example timing report:

```
Startpoint: reg_A Endpoint: reg_B Path Delay: 3.5 ns Slack: -0.23 ns
```

AI-generated summary:

```
Critical path detected between reg_A and reg_B Slack: -0.23 ns
Possible cause: long combinational logic Suggested fix: add pipeline register
```

Benefits of AI-based analysis:

Faster debugging

Automatic report summarization

Reduced manual effort

Improved engineering productivity

3.3 AI-Based Testbench Generation

Verification engineers often spend significant time writing testbenches to validate RTL modules. A testbench provides stimulus to the design and checks whether the outputs match the expected behavior.

However, writing testbenches manually is repetitive and requires understanding the functionality of each RTL module. For large designs containing hundreds of modules, creating verification environments becomes time-consuming.

Artificial Intelligence and Large Language Models (LLMs) can help automate testbench generation directly from RTL code.

The AI model analyzes the RTL module and automatically generates:

Input stimulus

Output observation signals

Simulation sequence

Basic test cases

Benefits of AI-based testbench generation include:

Faster verification setup

Reduced manual coding effort

Automatic generation of test scenarios

Improved verification productivity AI tools can also be extended to generate:

corner-case tests

randomized input sequences

functional coverage points

This approach significantly reduces the time required for verification engineers to create test environments

4. Industry Product Ideas

4.1 AI-Assisted RTL Design Platform

To address the challenges identified in RTL development and AI-generated code validation, we propose an AI-assisted RTL design platform.

The proposed system combines machine learning models with hardware design rules to analyze RTL code and detect common design issues automatically.

Key capabilities of the proposed platform include:

Automatic detection of RTL coding errors

Hardware-aware code suggestions

AI-based RTL generation

Automatic testbench generation

Explanation of synthesis warnings

By integrating AI assistance into the RTL design workflow, the platform can significantly reduce debugging time and improve the productivity of digital designers.

4.2 AI Timing Report Analyzer

A tool that parses STA reports and automatically identifies the worst slack paths and possible causes of timing violations.

4.3 AI Testbench Generator

A tool that generates testbenches automatically from RTL modules. Example workflow:

Input:Verilog RTL module

Output:Generated testbench,Stimulus generation,Output checking logic

Benefits:

Reduces verification time

Improves functional coverage